

Why LLMs Need to Change

The shelf life of a trained model, and the case for separating knowledge from computation

D.W. Murray / Sigmantics AB

Version 1.0 · 6 June 2026

Claims: an LLM's world-knowledge has a short, roughly annual shelf life because knowledge is fused to computation in the weights and cannot be cheaply updated, and the architecture should separate the two.

Does not claim: that LLMs are a dead end, that scaling or further capability gains are pointless, that capability has plateaued, or that a finished separated architecture already exists.

Methodology and source verification: AI Artifact Verification

The bargain we accepted

Every year, training a frontier model costs more. We accepted this without much argument, because every year the models could do things they could not do before. The bill was large, but it bought new capability, and new capability justified almost any number.

Epoch AI puts the trend at roughly 3.5x more training cost per year, with training compute growing about 5x per year and doubling every five months. The Stanford AI Index and Epoch place GPT-4 near 78 to 100 million dollars, Gemini Ultra near 192 million, and Llama 3.1 405B near 170 million. Frontier runs in 2026 sit between 200 and 500 million dollars, with credible projections of 1 to 3 billion for the late-2027 frontier.

Here is the part the bill hides. New capability is a one-time purchase. Currency is not. A second cost rides alongside the first on every run, and it has nothing to do with whether the model got smarter. It is the cost of the world having moved on since the last run. This piece is about that second cost: why it is structural rather than incidental, why neither a plateau nor continued progress removes it, and what has to change in how we build these systems to be rid of it.

The shelf life of a trained model

Run a thought experiment to strip the distraction away. Suppose capability plateaus tomorrow. Reasoning, coding, language, the core skills stop improving meaningfully run over run. We have the model we wanted.

We still cannot stop training.

A model trained on a 2025 corpus knows a 2025 world. Twelve months later it has the wrong head of state in some country, the wrong API for a popular library, the wrong tax thresholds, no idea a war started or ended, and a confident answer about a company that no longer exists. None of this is a capability failure. The model reasons as well as it ever did. It is simply current as of a date that keeps receding into the past.

So even at a full plateau, you retrain. Not to get smarter. Just to stay current.

Now drop the assumption, because it was only ever a device. Capability did not plateau in 2025 or 2026. Models kept clearing benchmarks that looked out of reach a year earlier. And the tax was paid the entire time, on top of the capability bill, not instead of it. That is what the plateau makes visible: currency is a separate cost that runs whether the frontier is moving or frozen. Capability is a one-time research win you could in principle amortize over many years. Currency is a perpetual tax. Progress does not pay it down. A plateau does not cancel it. It only removes the headline that was distracting you from it.

Say it plainly, because this is the whole argument. A trained model has a shelf life. The day training ends, the clock starts. Its picture of the world is frozen at the cutoff and drifts further from reality every day after. For anything that touches the present, the news, prices, people, laws, the state of the very tools the model is asked to use, that drift becomes disqualifying within about a year. Not because the model degraded. Because the world did not stop moving and the model did. A frontier model is a perishable good sold as durable goods, and the date stamped on the carton is roughly twelve months out.

The root cause: knowledge and computation are fused

The reason you cannot separate these two costs is that the architecture does not separate them either.

In every other piece of infrastructure we build, knowledge is addressable. A database row has a primary key. You update one fact with one write, in constant time, without touching any other fact. A codebase has named functions. You patch one and leave the rest alone. The knowledge in these systems lives in identifiable, mutable units.

In a dense transformer, knowledge has no address. What the model "knows" is distributed across billions of weights, entangled with everything else it knows and with the machinery of how it reasons. There is no row for the current tax code. There is no function you can patch. Knowledge and computation are the same tensor.

Three consequences follow, and they are the heart of the problem:

- You cannot query a model's knowledge as data and replace a slice of it.
- You cannot append. Adding new knowledge by gradient descent perturbs the entire weight set, which is what catastrophic forgetting is.
- Therefore the unit cost of integrating new knowledge is, in the limit, a full training run.

That last line is the whole argument. Everything below is evidence for it.

The three ways to update a model, and why only the expensive one works

There are exactly three things you can do to bring a model up to date. Two of them are cheap and do not work. The third works and is not cheap.

Edit the weights in place. This is the dream: locate the few parameters that store a fact and surgically rewrite them. Methods like ROME and MEMIT do exactly this, and at small scale they look impressive. At realistic scale they collapse. The WikiBigEdit benchmark shows ROME, R-ROME, and MEMIT degrading within the first few hundred edits, into outright model collapse. Even WISE, a method purpose-built for long edit streams, steadily decays and converges back to the model's pre-edit knowledge within roughly ten thousand updates, never beating plain retrieval. And the bar is lower than that number makes it sound. The point is not that the world produces more than ten thousand facts a year in some trivial counting sense. It is that the high-impact changes alone, the ones that actually break a model's usefulness (geopolitics, regulations, API and library churn, who runs which company) overwhelm these methods on their own. In-place editing is a demo, not a maintenance strategy.

Keep pretraining on new data. Continued pretraining is the obvious middle path, and it runs straight into catastrophic forgetting: teaching the model the new year quietly erodes the old years. As of 2026 there is no breakthrough here, though not for lack of effort. The continual-learning literature treats it as everywhere-discussed precisely because nobody has solved it, and the wave of mitigation work (replay schedules, model growth, regularization, low-perplexity masking) marks it as a live engineering priority, not a dormant one. The standing result has not moved: every method trades plasticity against stability. Forgetting is softened, never eliminated.

Retrain. Rebuild on a refreshed corpus. This is the only path that reliably produces a coherent, current model. It is also the path that does not amortize. You pay it, and then next year you pay it again, in full, for the same reason. There is no accumulation. The previous run does not lower the next one in any structural way.

Warm-starting from prior weights helps the arithmetic. Fine-tuning can run at 1 to 5 percent of a from-scratch run. But it reintroduces the forgetting risk, requires replaying old data to stay coherent, and in practice the frontier rebuilds anyway because of changing architectures and data mixes. The cost is recurring whether or not any single instance is literally a full run. Warm-starting changes the size of the recurring bill. It does not make the bill go away.

The reframe: this is a subscription, not an asset

Put the pieces together and the economic picture inverts.

We have been treating a training run as capital expenditure: a large, one-time investment in a durable asset. It is not. A trained model is a depreciating asset with a short half-life. Its capability layer might last, but its knowledge layer rots from the day training stops, and there is no cheap way to refresh just that layer because the architecture refuses to separate it. The recurring training cost is not investment. It is a subscription fee for the right to remain current, paid in full each cycle, forever.

That is the real problem with the paradigm, and it is independent of how good the models get. A perfect reasoner with a frozen, decaying knowledge base is still on the subscription. Scaling does not cancel it. It raises the monthly rate.

Three objections worth answering

A fair piece has to meet the obvious pushback head on. Three objections come up every time, and none of them rescues the paradigm. Each one, looked at squarely, ends up arguing for the same conclusion.

Retrieval (RAG). Fetch current facts from an external store at inference time instead of trusting the weights. Worth one line, not more: it works precisely by not trusting the weights for knowledge, which concedes the point rather than answering it. It patches what the model retrieves, never what the model is, so the frozen internal picture of the world stays exactly as stale. A useful band-aid, not a position. Set it aside.

Not all knowledge ages. True. Mathematics, logic, grammar, the fundamentals of code and physics are stable. Only the layer of entities, events, and current facts rots. So "ages fast" is precise only about one stratum. But that stratum is the one that matters for almost anything touching the present world, and fusion is exactly why you cannot isolate and refresh it on its own. The fact that the stable layer is stable is not a defense of the architecture. It is the strongest possible argument for separating the layers, because it proves they have different update frequencies and should not share a storage mechanism.

Cheaper, more frequent retrains. This is the real trend, and it deserves to be taken seriously rather than waved away. The cost of reaching a fixed level of capability is falling fast. Inference cost for a given level of performance has been roughly halving every couple of months, which means the very capability you are paying to refresh keeps getting cheaper to obtain. The per-cycle bill is on a downward trend even with no change to the architecture. Granted. But a cheaper subscription is still a subscription, and cost was never the core of the problem. The next section is why.

All three objections circle the same wall: knowledge that cannot be addressed and cannot be updated cheaply.

Even free retraining would not be enough

Push the cost trend to its limit. Suppose retraining became instant and free. Does the problem go away?

No, and seeing why is the whole case. Cost is the most visible symptom of the fusion, not the disease. The deeper deficits survive a price of zero:

- You still could not correct a single wrong fact without a global operation over the entire model. There is no targeted write.
- You still could not see what changed between two versions. A retrain hands you a new opaque weight set, not a diff.
- You still could not roll back one bad update. You get the next monolith or the previous one, and nothing in between.
- You still could not attribute a belief to its source. The model knows things and cannot tell you why, or from where.

None of these are cost problems. They are the properties any addressable store gives you for free and a fused one denies you at any price. Addressability, audit, rollback, provenance: a database hands you all four without being asked. A dense transformer refuses all four no matter how cheap the training run.

So the falling cost of retraining does not weaken the case for change. It strengthens it, by removing the last excuse. While retraining was ruinously expensive, you could call the architecture a regrettable necessity. Once it is cheap, the only thing still binding knowledge to computation is habit.

What needs to change

The change is not "abandon these models." It is "stop storing knowledge the way they currently store it."

The defensible, hard-to-argue version of the thesis is this. The dominant paradigm fuses knowledge and computation into a single non-addressable artifact. That fusion is the source of catastrophic forgetting, the failure of in-place editing, and the perpetual retraining tax. No method available today solves it at scale. And the economics of paying for currency forever compare badly against any architecture that keeps the two apart.

So the direction is separation of concerns, applied to cognition. Keep the computational substrate, the part that reasons, which genuinely is a durable, amortizable asset. Move the knowledge out of the weights and into a layer that is addressable, appendable, and inspectable: something you can write one fact to, audit, version, and roll back, the way we already do with every other system that has to stay current. The mature version makes the knowledge layer a first-class, structured, queryable store, and makes the boundary between what the system can do and what the system currently believes about the world explicit in the architecture instead of smeared across a tensor.

This is harder than one sentence makes it sound, and it would be dishonest to pretend otherwise. The entanglement that causes the aging problem may also be where some of the model's most valuable behavior lives. In-context learning, analogical transfer, the rough coherence of its world model: these emerge from knowledge and computation sharing one representational space, and a clean split risks losing them. The broader idea, a knowledge store feeding a separate reasoner, is also not new. It is most of the history of AI before deep learning, and that lineage never scaled to frontier-level fluency. So this is a research direction, not a finished blueprint. Today's retrieval-and-agent stacks are at best a crude prototype of it. The mature form does not exist yet at scale. Naming the destination is not the same as arriving. But the destination is clear, and the present architecture is provably not it.

A model built that way does not age. Its reasoning is a fixed asset. Its knowledge is data, and data you can update one row at a time.

That is the change. Not a smarter monolith. A system where being current costs a write, not a training run.

References

1. Epoch AI. *Trends in Artificial Intelligence*. Training cost rising ~3.5x per year; training compute ~5x per year, doubling every ~5 months; inference cost for a fixed performance level halving roughly every two months. <https://epoch.ai/trends>
 2. Epoch AI. *How Much Does It Cost to Train Frontier AI Models?* Frontier training-cost trajectory and billion-dollar-run projections. <https://epoch.ai/blog/how-much-does-it-cost-to-train-frontier-ai-models>
 3. Stanford HAI. *AI Index Report 2025*. Training-cost estimates for GPT-4, Gemini Ultra, and Llama 3.1 405B. <https://hai.stanford.edu/ai-index>
 4. WikiBigEdit: *Understanding the Limits of Lifelong Knowledge Editing in LLMs* (2025). arXiv:2503.05683. ROME, R-ROME, and MEMIT collapse within a few hundred sequential edits; WISE decays back toward pre-edit knowledge within ~10k updates and does not beat retrieval. <https://arxiv.org/abs/2503.05683>
 5. Gupta, Rao, Anumanchipalli, et al. *Model Editing at Scale Leads to Gradual and Catastrophic Forgetting* (2024). arXiv:2401.07453. Disabling edits as a fundamental limitation of locate-and-edit methods. <https://arxiv.org/abs/2401.07453>
-